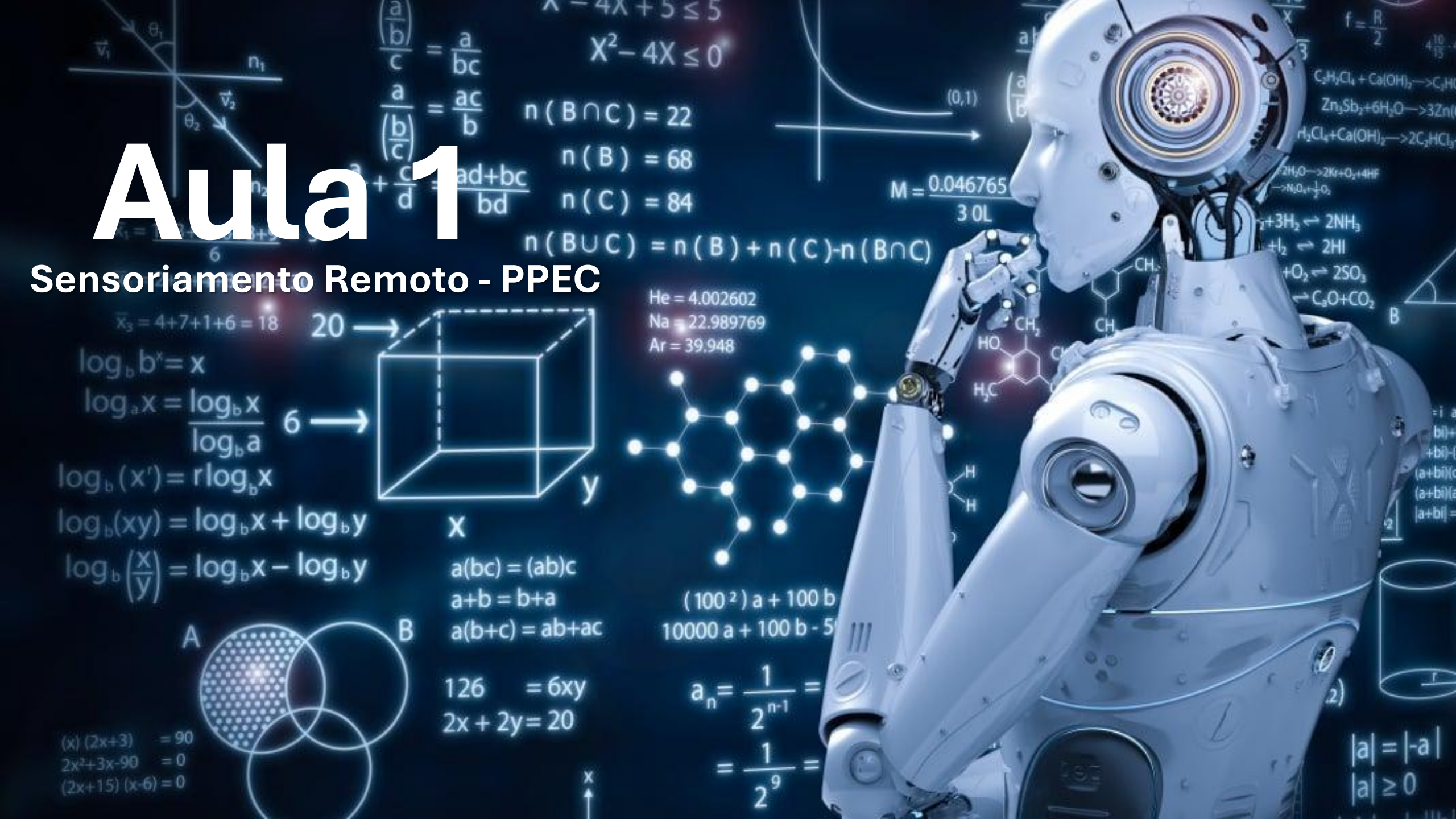


Aula 1

Sensoriamento Remoto - PPEC



$$\vec{v}_1$$
$$\theta_1$$
$$n_1$$
$$\vec{v}_2$$
$$\theta_2$$
$$n_2$$
$$x_1 = 1$$
$$x_2 = 2$$
$$x_3 = 4$$
$$x_4 = 7$$
$$x_5 = 1$$
$$x_6 = 6$$
$$\bar{x}_3 = 4 + 7 + 1 + 6 = 18$$

$$\frac{\left(\frac{a}{b}\right)}{c} = \frac{a}{bc}$$
$$\frac{a}{\left(\frac{b}{c}\right)} = \frac{ac}{b}$$
$$\frac{a}{\left(\frac{b}{c}\right)} + \frac{c}{d} = \frac{ad+bc}{bd}$$

$$X^2 - 4X \leq 0$$
$$n(B \cap C) = 22$$
$$n(B) = 68$$
$$n(C) = 84$$
$$n(B \cup C) = n(B) + n(C) - n(B \cap C)$$

$$(0,1)$$
$$M = \frac{0.046765}{30L}$$



$$f = \frac{R}{2}$$
$$4 \frac{10}{15}$$
$$i = a$$
$$bi +$$
$$+ bi) -$$
$$(a + bi)(c$$
$$(a + bi)(a$$
$$|a + bi| =$$
$$|a| = |-a|$$
$$|a| \geq 0$$

$$\log_b b^x = x$$
$$\log_a x = \frac{\log_b x}{\log_b a}$$
$$\log_b(x^r) = r \log_b x$$
$$\log_b(xy) = \log_b x + \log_b y$$
$$\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y$$

$$a(bc) = (ab)c$$
$$a+b = b+a$$
$$a(b+c) = ab+ac$$
$$126 = 6xy$$
$$2x + 2y = 20$$

$$He = 4.002602$$
$$Na = 22.989769$$
$$Ar = 39.948$$
$$(100^2)a + 100b$$
$$10000a + 100b - 5$$
$$a_n = \frac{1}{2^{n-1}} =$$
$$= \frac{1}{2^9} =$$



$$|a| = |-a|$$
$$|a| \geq 0$$

$$|a| = |-a|$$
$$|a| \geq 0$$

**ONDE HÁ
HUMILDADE PARA
ACEITAR OS ERROS,
TAMBÉM HAVERÁ
INTELIGÊNCIA PARA
APRENDER COM
ELES.**

Sensoriamento Remoto

Todos os dias, centenas de satélites de observação da Terra capturam petabytes de imagens enquanto orbitam o planeta. O Sentinel-2 sozinho entrega mais de 1,6 terabytes de dados multiespectrais diariamente, cobrindo toda a superfície terrestre a cada cinco dias.

Enquanto isso, constelações comerciais da Planet Labs e Vantor imageam cada ponto da Terra em resolução de aproximadamente três metros a submétrica todos os dias. O resultado é uma inundação sem precedentes de informações geoespaciais que nenhuma equipe de analistas humanos poderia esperar examinar manualmente.

Sensoriamento Remoto

A lacuna entre nossa capacidade de coletar dados e nossa capacidade de extrair informações úteis deles nunca foi tão grande.

Sensoriamento Remoto

É aqui que a inteligência artificial muda a equação.

Em 2022, a **Microsoft** lançou um dataset global de pegadas de edificações contendo mais de 1,2 bilhão de estruturas, extraídas automaticamente de imagens de satélite usando deep learning.

O projeto Dynamic World do **Google** classifica a cobertura do solo em todo o planeta em tempo quase real, atualizando cada vez que uma nova imagem do Sentinel-2 se torna disponível 10 .

O *Segment Anything Model* (SAM) da **Meta** demonstrou que um único modelo fundamental poderia segmentar virtualmente qualquer objeto em qualquer imagem.

Sensoriamento Remoto e a GeoAI

Esses avanços compartilham um fio comum: estão na interseção entre ciência de dados geoespaciais e inteligência artificial, um campo em rápido crescimento conhecido como GeoAI.

O Que é GeoAI?

GeoAI refere-se à integração da ciência de dados geoespaciais com inteligência artificial, particularmente aprendizado de máquina e deep learning, para analisar, interpretar e gerar insights a partir de dados espaciais. Ele combina conhecimento de domínio sobre sensoriamento remoto e cartografia com as capacidades de reconhecimento de padrões dos sistemas modernos de IA.

Por Que Dados Espaciais São Diferentes?

Uma percepção crítica que distingue o GeoAI da visão computacional geral é que dados geoespaciais são fundamentalmente diferentes de imagens comuns.

- ✓ Sistemas de referência de coordenadas e projeções.
- ✓ Resolução espacial e escala.
- ✓ Bandas espectrais.
- ✓ Dimensão temporal.
- ✓ Autocorrelação espacial.
- ✓ Formatos diversos.
- ✓ Tamanho de arquivos e processamento em tiles.

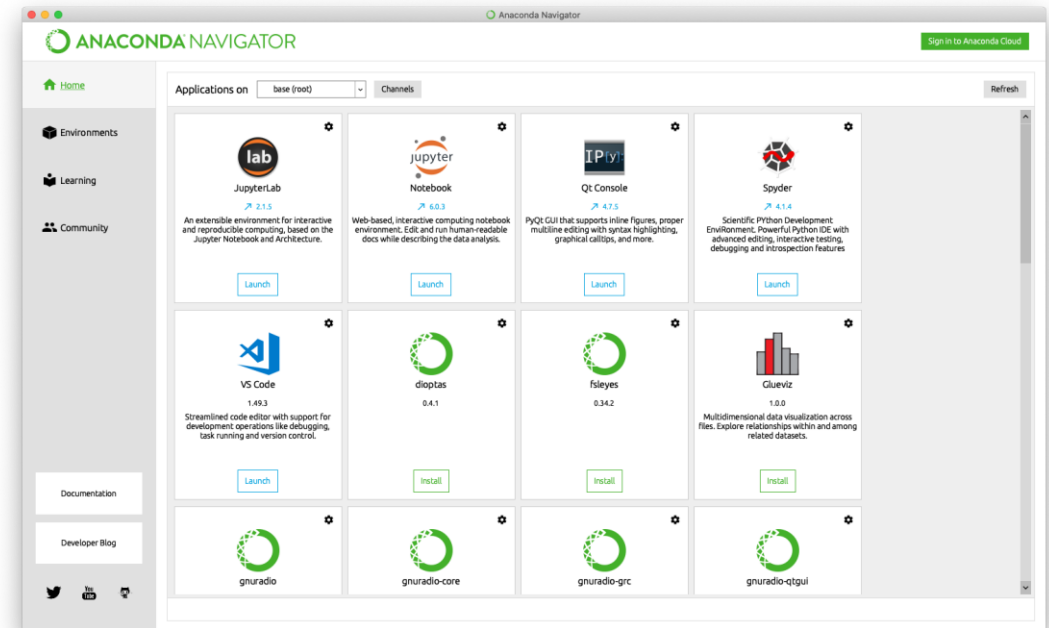
Sobre o Python

O **Python** é uma linguagem de programação de alto nível, interpretada e de propósito geral. Criada por Guido van Rossum e lançada em 1991, ela é muito famosa por sua facilidade de leitura e escrita, lembrando o idioma inglês, o que a torna excelente para quem está começando a programar.



ANACONDA

O **Anaconda Python** é um pacote livre que junta a linguagem Python com mais de 1.500 ferramentas úteis para ciência de dados, inteligência artificial e matemática.



Onde utilizar...

Editores de Código

VS Code

PyCharm

Positron



Ambientes de Dados

Jupyter Notebook

Google Colab

Spyder



SPYDER

Softwares com
Python Embutido

Autodesk Maya

QGIS / ArcGIS

FreeCAD

GIMP

ENVI



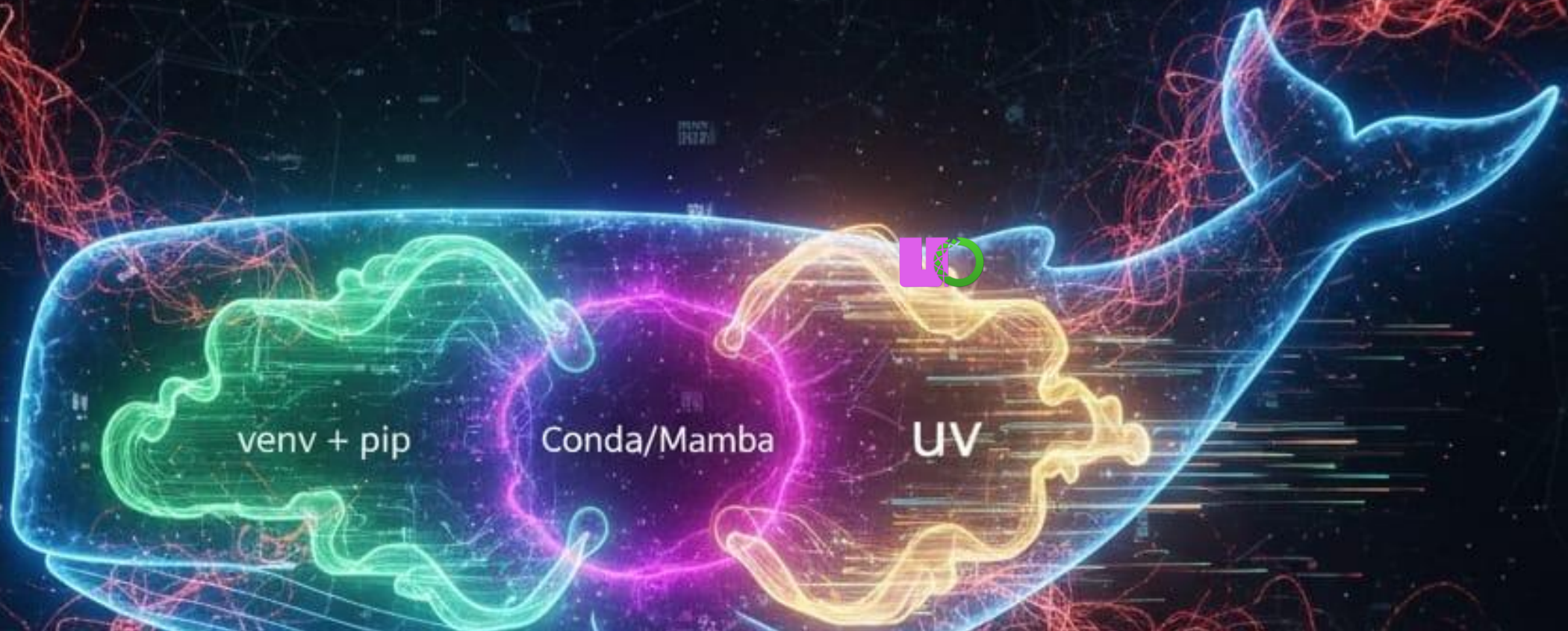
O que são bibliotecas de código?

Imagine que você vai construir uma casa. Em vez de fabricar os tijolos, os parafusos e as portas do zero, você vai até uma loja de materiais de construção e compra essas peças prontas. Na programação, as bibliotecas funcionam exatamente assim: elas evitam que você precise "reinventar a roda" a cada novo programa.



Métodos de instalação

Os gerenciadores de pacotes Python (**pip**, **conda**, **mamba** e **uv**) servem para instalar bibliotecas, mas operam de formas totalmente diferentes. O **uv** e o **mamba** são alternativas modernas e muito mais rápidas ao **pip** e ao **conda**, respectivamente.



Ambiente virtual



O venv é uma ferramenta padrão do Python usada para criar ambientes virtuais isolados. Cada ambiente possui sua própria pasta com uma cópia do interpretador Python e seu próprio gerenciador de pacotes (pip), permitindo instalar bibliotecas para um projeto específico sem bagunçar o sistema operacional ou outros.

```
python -m venv venv
```

Criar

```
venv\Scripts\activate
```

Ativar

```
deactivate
```

Desativar

Instalação do geoai-py

```
>> pip install geoai-py
```

```
>> conda install -c conda-forge geoai
```

```
blems  Output  Debug Console  Terminal  Ports

>S C:\Users\mauro>
>>
>> pip install geoai-py
>>
 28.6/28.6 MB 5.9 MB/s 0:00:04
Downloading transformers-5.15.1-py3-none-any.whl (11.7 MB)
 11.7/11.7 MB 6.4 MB/s 0:00:01
Downloading tokenizers-0.22.2-cp39-abi3-win_amd64.whl (2.7 MB)
 2.7/2.7 MB 5.8 MB/s 0:00:00
Downloading regex-2026.7.19-cp314-cp314-win_amd64.whl (280 kB)
Downloading safetensors-0.8.0-cp310-abi3-win_amd64.whl (355 kB)
Downloading alumentations-2.0.8-py3-none-any.whl (369 kB)
Downloading albucore-0.0.24-py3-none-any.whl (15 kB)
Downloading opencv_python_headless-5.0.0.93-cp37-abi3-win_amd64.whl (43.8 MB)
 43.8/43.8 MB 6.5 MB/s 0:00:06
Downloading simsimd-6.5.16-cp314-cp314-win_amd64.whl (90 kB)
Downloading stringzilla-5.1.2-cp314-cp314-win_amd64.whl (884 kB)
 884.7/884.7 kB 7.3 MB/s 0:00:00
Downloading contextily-1.7.1-py3-none-any.whl (18 kB)
Downloading geopy-2.5.0-py3-none-any.whl (114 kB)
Downloading geographiclib-2.1-py3-none-any.whl (40 kB)
Downloading jupyter_server_proxy-4.5.0-py3-none-any.whl (45 kB)
Downloading simpervisor-1.0.0-py3-none-any.whl (8.3 kB)
Downloading localtileserver-1.0.0-py3-none-any.whl (33.7 MB)
 30.7/33.7 MB 7.3 MB/s eta 0:00:01
```



A instalação pode demorar uns 15 min...

Deep Learning : PyTorch, TensorFlow

Deep Learning (Aprendizado Profundo) é uma subárea do Aprendizado de Máquina (*Machine Learning*) focada no treinamento de redes neurais artificiais com múltiplas camadas. Essas redes aprendem a reconhecer padrões complexos em grandes volumes de dados (como imagens, áudio, texto ou dados tabulares), imitando intuitivamente a forma como os neurônios biológicos processam informações.

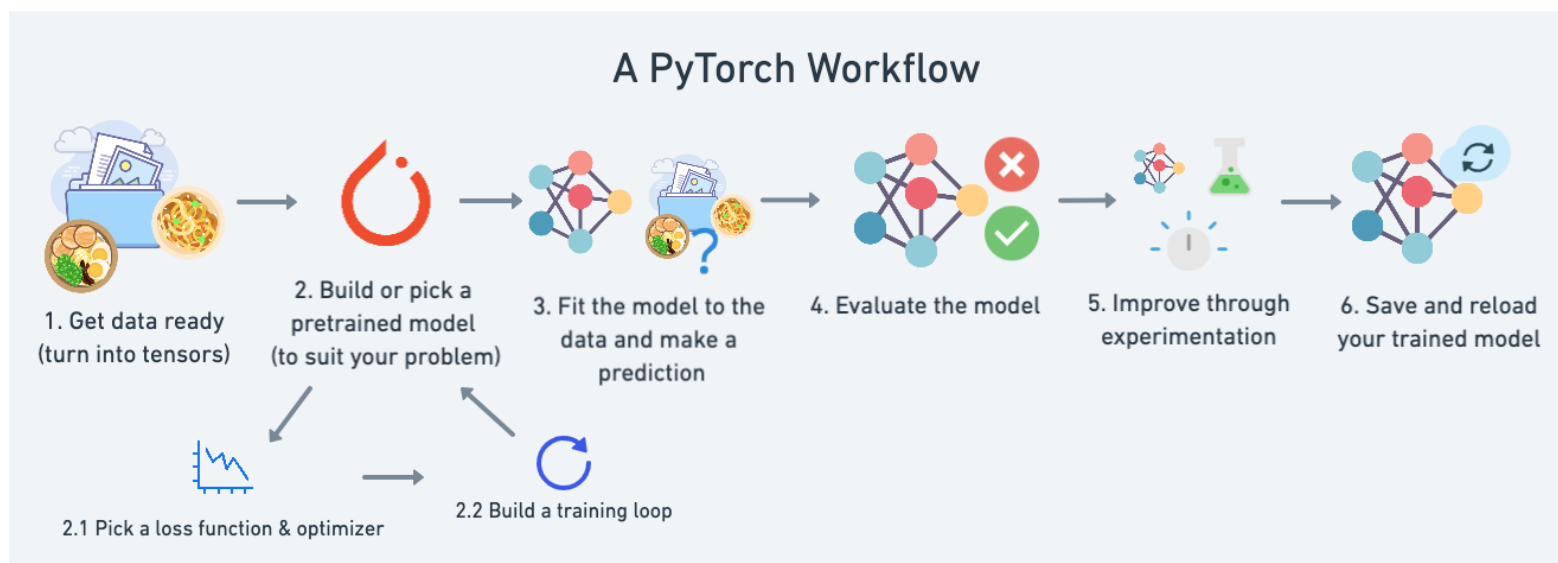


Em Python, as duas principais ferramentas para construir e treinar essas redes neurais são o **PyTorch** e o **TensorFlow**.

Como funciona o PyTorch?



O **PyTorch** se destaca pela flexibilidade. Por usar grafos computacionais dinâmicos, a rede é construída à medida que o código executa. Isso facilita a depuração com ferramentas padrão do Python (como o pdb ou print) e o manuseio de arquiteturas complexas ou de tamanho variável.



Python

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. Definição da Arquitetura
class MinhaRede(nn.Module):
    def __init__(self):
        super().__init__()
        self.camada_oculta = nn.Linear(in_features=10, out_features=64)
        self.camada_saida = nn.Linear(in_features=64, out_features=1)
        self.relu = nn.ReLU()

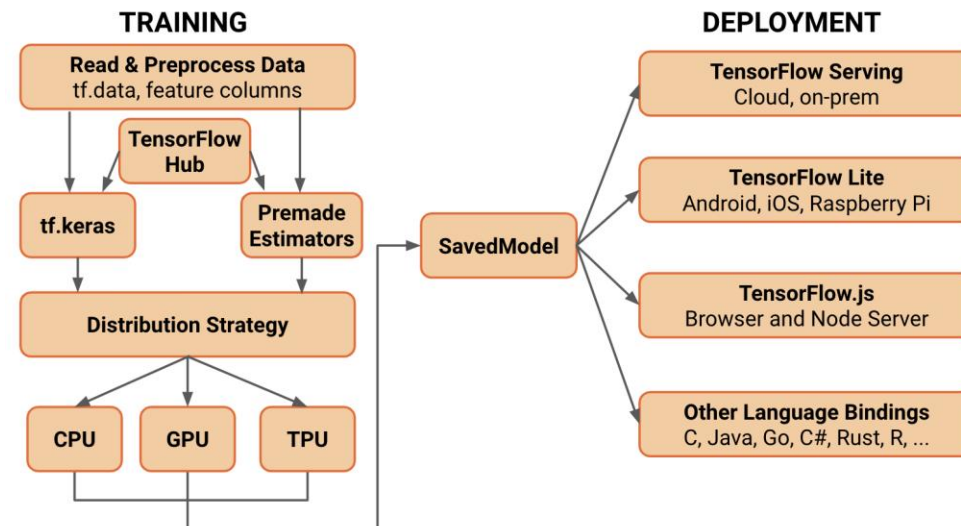
    def forward(self, x):
        x = self.relu(self.camada_oculta(x))
        x = self.camada_saida(x)
        return x

modelo = MinhaRede()
criterio = nn.MSELoss()
otimizador = optim.Adam(modelo.parameters(), lr=0.001)

# 2. Loop de Treinamento Explícito
# x_dados, y_dados = ...
# otimizador.zero_grad()           # Zera os gradientes acumulados
# predicoes = modelo(x_dados)      # Passo forward
# perda = criterio(predicoes, y_dados) # Calcula o erro
# perda.backward()                  # Passo backward (Backpropagation)
# otimizador.step()                 # Atualiza os pesos
```

Como funciona o TensorFlow (com Keras)?

O **TensorFlow** foi desenvolvido focado em desempenho e implantação em larga escala. Integrado nativamente ao **Keras** (sua API de alto nível), ele permite criar modelos de forma modular e altamente legível, além de oferecer ecossistemas robustos para publicação (como *TensorFlow Serving*, *TF Lite* para dispositivos móveis e *TensorFlow.js* para navegadores).



Python

```
import tensorflow as tf
from tensorflow.keras import layers, models

# 1. Definição Sequencial do Modelo
modelo = models.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(1)
])

# 2. Compilação (Define o otimizador, função de perda e métricas)
modelo.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# 3. Treinamento em uma única linha
# modelo.fit(x_dados, y_dados, epochs=10, batch_size=32)
```


Característica	PyTorch	TensorFlow (Keras)
Desenvolvedor Principal	Meta (Facebook) AI Research	Google
Estilo de Grafo	Dinâmico (Define-by-run)	Estático/Híbrido (Define-and-run, via tf.function)
Sintaxe/Estilo	Sensação nativa em Python (Pythônica)	Mais declarativa e estruturada
Uso Principal	Pesquisa acadêmica e prototyping rápido	Produção industrial e sistemas de grande escala
API de Alto Nível	PyTorch Lightning, torch.nn	tf.keras

Fundamentos de ML

- **Treinamento:** É a etapa de aprendizado em que o modelo analisa os dados, calcula o erro e ajusta seus pesos internos (parâmetros) por meio de algoritmos de otimização (como a retropropagação). É um processo computacionalmente pesado e demorado.
- **Inferência:** É a fase de aplicação prática. Com os pesos congelados, o modelo apenas recebe novos dados e gera previsões/classificações em tempo real. Exige muito menos poder computacional.

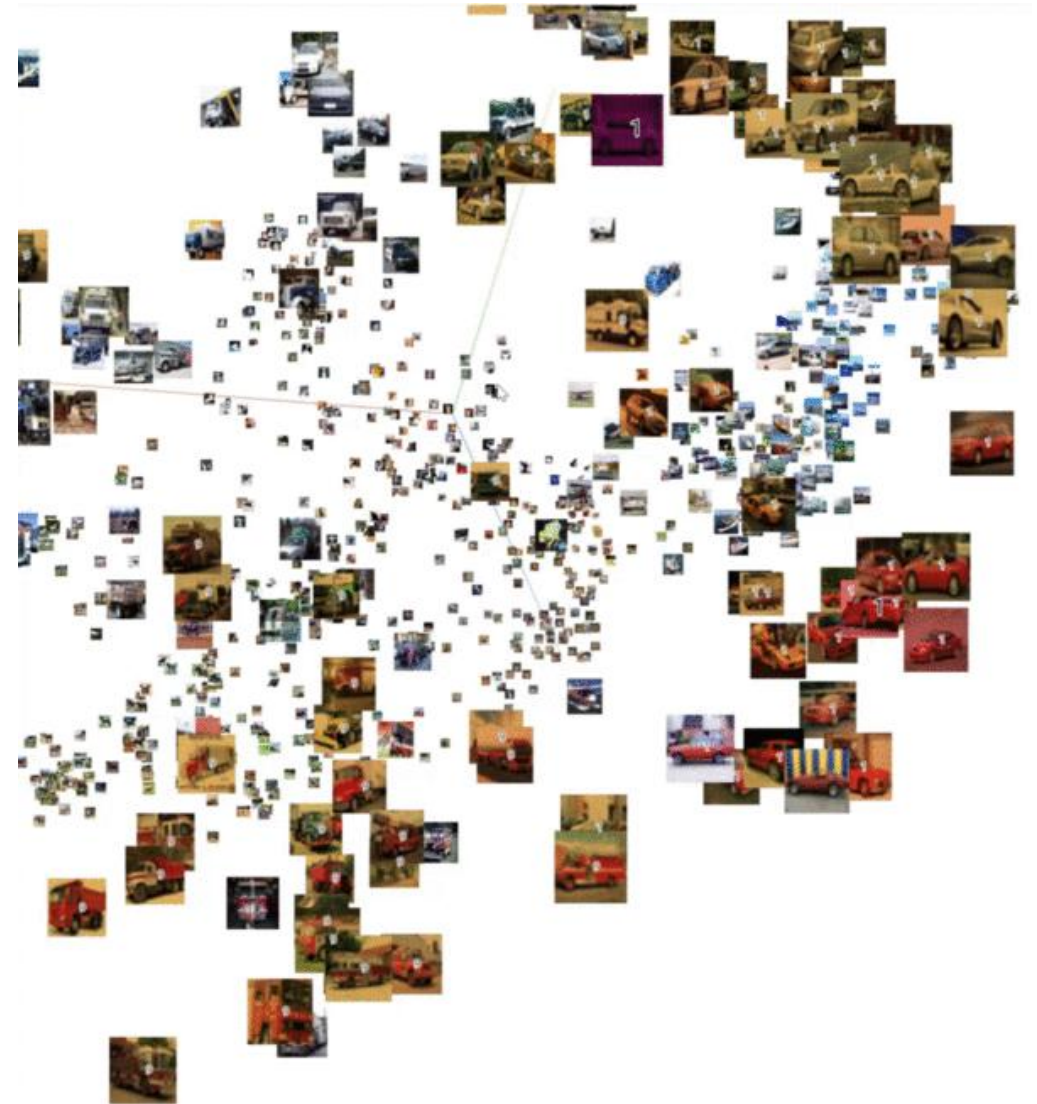
Fundamentos de ML: **Embedding**

- **Entrada dos pixels:** A imagem começa como uma matriz de pixels de cores (RGB).
- **Extração de características visuais:** A imagem passa por uma rede neural profunda — como uma **CNN** (Rede Neural Convolutacional) ou um **Vision Transformer (ViT)**.
 - As primeiras camadas identificam elementos simples, como bordas, cantos e texturas.
 - As camadas intermediárias agrupam essas formas em partes de objetos (como olhos, rodas ou folhas).
 - As camadas finais combinam essas partes para entender o contexto global (ex: "um gato deitado no sofá").
- **Redução e vetorização:** Em vez de usar a saída final de classificação (que diria apenas "gato"), o modelo extrai a representação da penúltima camada da rede. O resultado é um vetor denso contendo o significado visual comprimido.

Fundamentos de ML: Embedding

A grande utilidade dos embeddings é que imagens visualmente ou conceitualmente semelhantes ficam **próximas umas das outras** dentro do espaço multidimensional.

- **Similaridade de Cosseno ou Distância Euclidiana:** Para saber se duas imagens são parecidas, calcula-se a distância matemática entre seus vetores. Se o ângulo entre os vetores for pequeno, o conteúdo das imagens é semanticamente similar.
- **Busca Multimodal :** Redes como o **CLIP (OpenAI)** são treinadas para colocar texto e imagens no *mesmo* espaço de embedding.



Fundamentos de ML

Overfitting (Superajuste) Ocorre quando o modelo "decora" os dados de treinamento em vez de aprender os padrões gerais. Ele apresenta altíssimo desempenho no conjunto de treino, mas erra bastante ao tentar prever novos dados (dados de teste).

Como diagnosticar: O erro no treino continua caindo, enquanto o erro na validação começa a subir.

Como mitigar: Utilizar mais dados, aplicar técnicas de **regularização** (L1/L2, Dropout), simplificar a arquitetura da rede ou interromper o treino precocemente (*Early Stopping*).

Fundamentos de ML

Métricas de Avaliação

A escolha da métrica depende do tipo de problema (classificação ou regressão):

- 1. Para problemas de Classificação**
- 2. Para problemas de Regressão**

Para problemas de Classificação

Acurácia: Percentual total de acertos. Útil em dados balanceados, mas enganosa em conjuntos desbalanceados.

Precisão: Das previsões positivas que o modelo fez, quantas estavam corretas?

$$\left(\frac{TP}{TP+FP}\right)$$

Recall (Sensibilidade): Dos casos reais positivos, quantos o modelo conseguiu encontrar?

$$\left(\frac{TP}{TP+FN}\right)$$

F1-Score: Média harmônica entre Precisão e Recall. Ideal para encontrar um equilíbrio em cenários desbalanceados.

Matriz de Confusão: Tabela cruzada mostrando Verdadeiros Positivos (TP), Verdadeiros Negativos (TN), Falsos Positivos (FP) e Falsos Negativos (FN).

Para problemas de Regressão

MAE (Erro Médio Absoluto): Média das diferenças absolutas entre o valor real e a previsão.

MSE (Erro Quadrático Médio): Média dos erros ao quadrado; penaliza erros grandes de forma mais severa.

RMSE (Raiz do Erro Quadrático Médio): Raiz quadrada do MSE, retornando o erro para a mesma unidade da variável de saída.

R^2 (Coeficiente de Determinação): Indica o quanto a variação dos dados é explicada pelo modelo (varia de 0 a 1).

Tarefas Principais de IA na Análise Geoespacial

Compreender essas tarefas conceitualmente é essencial antes de mergulhar na implementação.

Reconhecimento de Imagens

O reconhecimento atribui um único rótulo categórico a uma imagem inteira ou chip de imagem. Dado um tile de imagem de satélite, o modelo prevê uma categoria em nível de cena como “residencial,” “industrial,” “agrícola” ou “florestal”



Arquiteturas como **ResNet**, **EfficientNet** e **Vision Transformers (ViT)** são comumente usadas para esta tarefa.

Detecção de Objetos

A detecção de objetos foca em encontrar e localizar objetos discretos dentro de uma imagem. Em vez de classificar cada pixel, modelos de detecção desenharam caixas delimitadoras ao redor de objetos de interesse e atribuem a cada caixa um rótulo de classe e uma pontuação de confiança



Modelos como Faster **R-CNN**, **YOLO** e **DETR** foram adaptados para imagens aéreas.

Segmentação Semântica

A segmentação semântica é a tarefa de atribuir um rótulo de classe a cada pixel em uma imagem. Dada uma imagem de satélite, um modelo de segmentação pode rotular cada pixel como edificação, estrada, vegetação, água ou solo exposto



Arquiteturas como **U-Net**, com estruturas codificador-decodificador e conexões de atalho, tornaram-se uma escolha padrão para segmentação geoespacial.

Segmentação de Instâncias

A segmentação de instâncias combina as forças da segmentação semântica e da detecção de objetos.



Mask **R-CNN** e suas variantes são as arquiteturas mais comuns para segmentação de instâncias. No domínio geoespacial, o **Segment Anything Model (SAM)** também provou ser altamente eficaz na produção de máscaras em nível de instância

Tradução de Imagens

A tradução de imagens é a tarefa de transformar uma imagem de um domínio ou representação para outro, preservando sua estrutura espacial. No contexto geoespacial, a aplicação mais proeminente é a super-resolução.



Arquiteturas comumente usadas para esta tarefa incluem **SRCNN**, **ESRGAN** e, mais recentemente, abordagens baseadas em difusão que produzem saídas nítidas e realistas.

Detecção de Mudanças

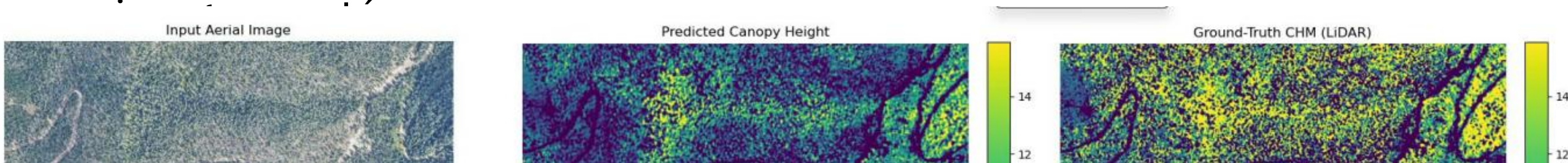
A detecção de mudanças identifica diferenças entre imagens do mesmo local capturadas em momentos diferentes. A forma mais simples compara duas imagens e produz um mapa binário indicando onde a mudança ocorreu.



Modelos para detecção de mudanças utilizam **redes siamesas** ou arquiteturas de **codificador duplo** que aprendem a extrair e comparar feições dos dois momentos temporais.

Regressão de Pixels

Nem todas as previsões geoespaciais são categóricas. A regressão de pixels prevê um valor contínuo para cada pixel em vez de uma classe discreta. As aplicações incluem estimativa de altura do dossel a partir de imagens de satélite (veja Figure 7), previsão de biomassa acima do solo, mapeamento de umidade do solo ou geração de modelos digitais de elevação a partir de pares de



Modelos de regressão usam arquiteturas similares àquelas para segmentação (**redes codificador-decodificador**), mas substituem a **cabeça de classificação** por uma **cabeça de regressão** que gera valores contínuos.

De Métodos Tradicionais a Modelos Fundamentais

Aprendizado de máquina tradicional: Random forests.

Redes neurais convolucionais: Arquiteturas como U-Net, DeepLab e Feature Pyramid Networks.

Modelos fundamentais: modelos grandes pré-treinados em datasets massivos. SAM da Meta, Prithvi da NASA e IBM, Clay, SatCLIP, AlphaEarth, SkySense++ e TESSERA.

Modelos de visão-linguagem: adicionam outra dimensão ao conectar compreensão visual com linguagem natural. Modelos como CLIP.

Agentes de IA: representam uma fronteira emergente, combinando múltiplos modelos, ferramentas e capacidades de raciocínio para lidar com fluxos de trabalho geoespaciais complexos e multi-etapas de forma autônoma.

Exercício 1: Explorando o GeoAI no Mundo Real

Escolha um dos seis domínios de aplicação discutidos neste capítulo (planejamento urbano, agricultura, monitoramento ambiental, resposta a desastres, gestão de recursos hídricos ou saúde pública). Pesquise um projeto ou produto específico do mundo real que usa GeoAI nesse domínio. Alguns pontos de partida para sua pesquisa incluem Microsoft's Global Building Footprints ²⁷, Google's Open Buildings ²⁸, Meta's High-Resolution Canopy Height Maps ²⁹ ou Google's Flood Hub ³⁰. Aborde as seguintes questões:

- Que problema o projeto resolve?
- Que tipo de dados geoespaciais ele usa (imagens de satélite, fotos aéreas, LiDAR, SAR, etc.)?
- Em qual das sete tarefas principais de IA ele se baseia primariamente?

Exercício 2: Comparando Dados Espaciais e Não Espaciais

Considere uma foto padrão do ImageNet de um cachorro e uma imagem de satélite Sentinel-2 de 10 metros de uma área urbana. Liste pelo menos sete diferenças específicas entre esses dois tipos de dados que afetariam como você projeta e treina um modelo de IA. Para cada diferença, explique brevemente por que ela importa para o design do modelo, treinamento ou avaliação.

Exercício 3: Associando Tarefas a Problemas

Para cada uma das sete tarefas principais de IA introduzidas neste capítulo (reconhecimento de imagens, detecção de objetos, segmentação semântica, segmentação de instâncias, tradução de imagens, detecção de mudanças e regressão de pixels), descreva um problema geoespacial concreto que seria melhor abordado por essa tarefa. Explique por que a tarefa escolhida é mais apropriada do que as alternativas. Por exemplo, por que a extração de pegadas de edificações usaria segmentação de instâncias em vez de segmentação semântica?